

第12课 云原生的 关键系统介绍

北京交通大学软件学院

严禁复制



目录



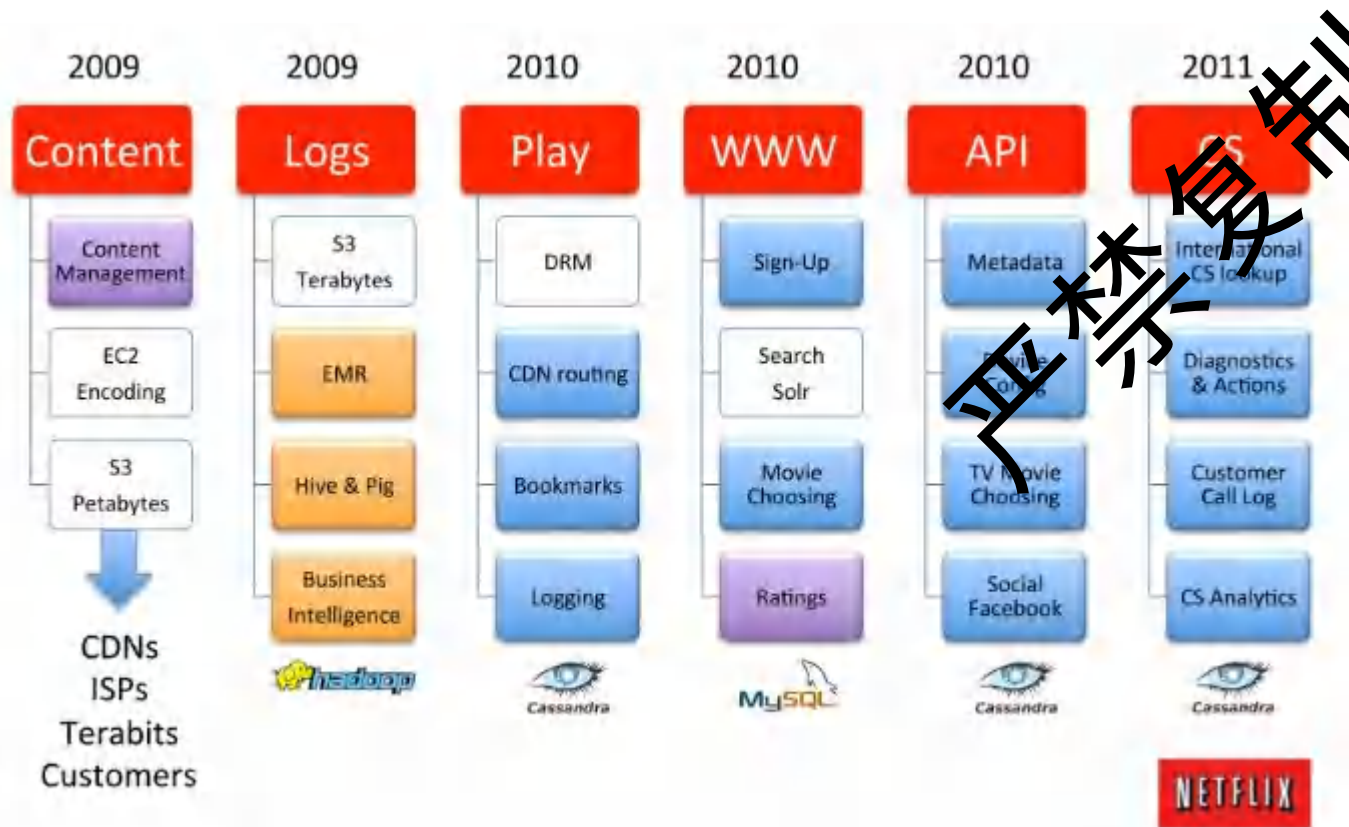
01. 云原生概述
02. 快速开发
03. 容器化部署
04. 微服务编排
05. 日志和监控



01 复制 原生概述

Netflix的微服务发展

- 微服务助力Netflix高速发展



2025, 60亿小时/周, 每个人1小时/周



普通架构 VS 云原生架构

Netflix Datacenter vs. Cloud Arch

Anti-Architecture

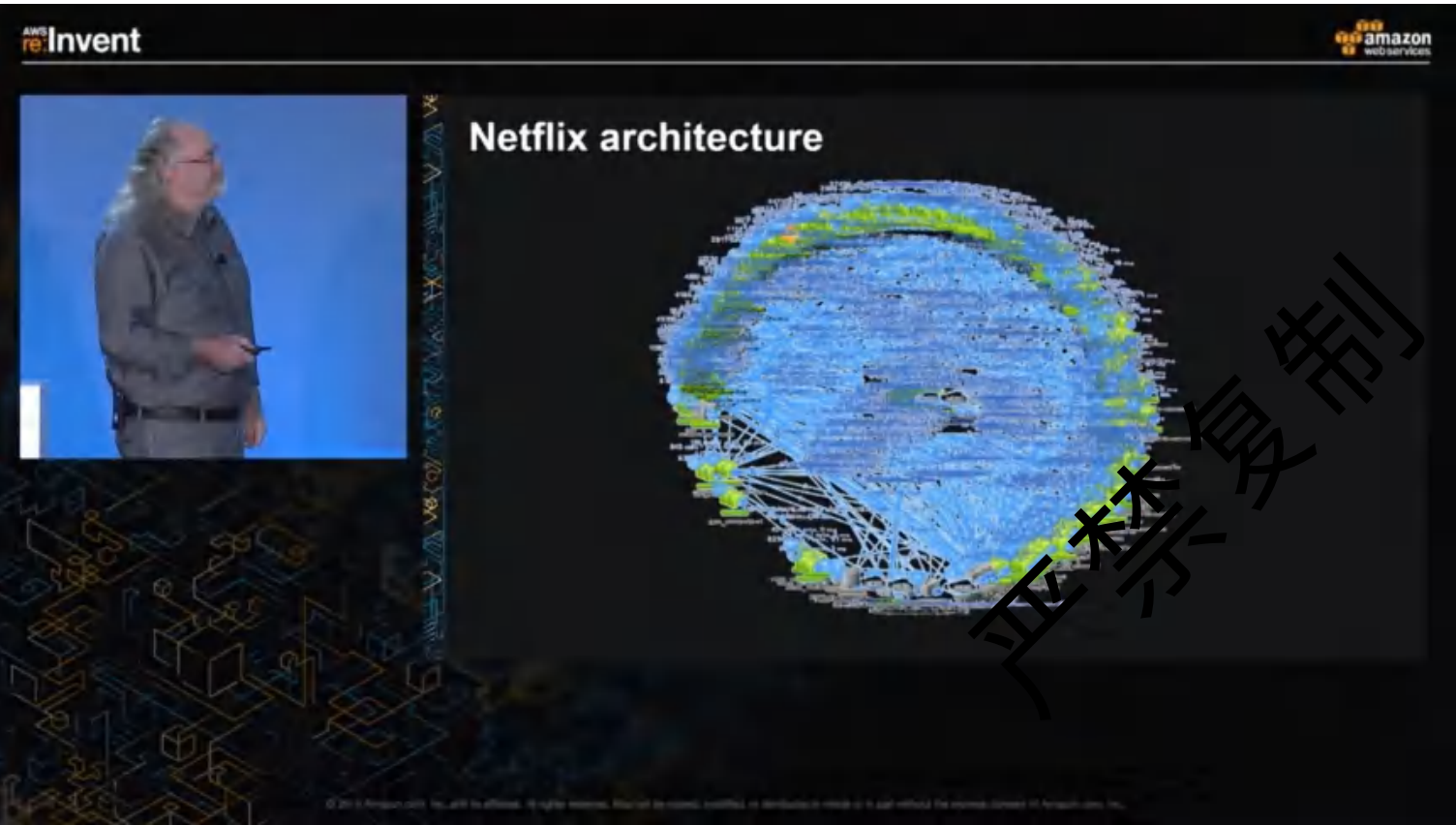
Central SQL Database	Distributed Key/Value NoSQL
Sticky In-Memory Session	Shared Memcached Session
Chatty Protocols	Latency Tolerant Protocols
Tangled Service Interfaces	Layered Service Interfaces
Instrumented Code	Instrumented Service Patterns
Fat Complex Objects	Lightweight Serializable Objects
Components as Jar Files	Components as Services

微服务的另一面

aws re:Invent

amazon web services

Netflix architecture



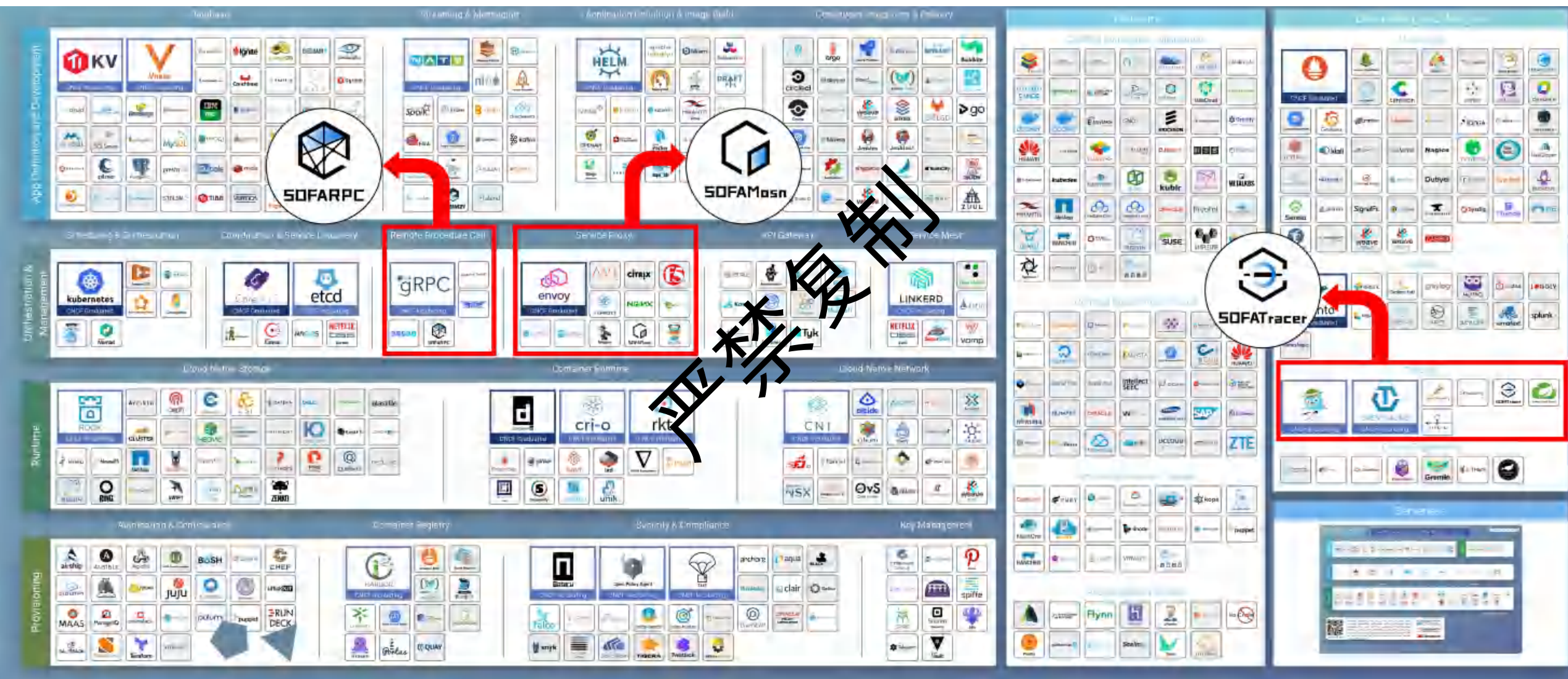
© 2014 Amazon.com, Inc. or its affiliates. All rights reserved. Amazon and the Amazon logo are trademarks of Amazon.com, Inc. or its affiliates. All other marks contained herein are the property of their respective owners.



云原生：拥抱云的分布式系统哲学

- 什么是云原生？
 - 一种构建和运行应用程序的方法，充分利用云计算模型的优势。
 - 云计算模型=通过网络共享计算资源（按需使用） 例如：IaaS, PaaS, SaaS
- CNCF (Cloud Native Computing Foundation)
 - 定义： 包含容器、微服务、不可变基础设施、声明式API等。
- 核心理念：
 - 弹性、韧性、可观测性、自动化、快速交付

云原生技术栈



学习路径 1-5

• 容器化

- Docker, 任意大小的应用程序和依赖项进行容器化。

• CI/CD (持续集成和持续发布)

• 应用编排

- Kubernetes
- Helm Charts升级Kubernetes上运行的应用。

• 监控和分析

- 将Prometheus用于监控、Fluentd用于日志、
- Jaeger用于整个应用调用链的跟踪。

• 服务代理、发现和治理

- CoreDNS 用于服务发现和服务治理,
- Envoy提供服务的健康检查
- Linkerd提供请求路由、和负载均衡等功能。



CLOUD NATIVE TRAIL MAP

The Cloud Native Landscape contains a large number of options. This Cloud Native Trail Map is a recommended practice for navigating this landscape. Cloud Native technologies. At each step, you can choose a vendor-neutral offering or use a vendor, and everything after step 5 is optional based on your circumstances.

HELP ALONG THE WAY

A. Training and Certification

Consider taking courses from CNCF and then take the exam to become a Certified Kubernetes Administrator or a Certified Kubernetes Application Developer.

B. Getting Help

Visit the CNCF website for community resources, including a mailing list, chat, and a Slack channel. You can also find a local CNCF chapter.

C. Join CNCF's End User Community

For companies that don't offer cloud native services internally, visit the CNCF website.

WHAT IS CLOUD NATIVE?

Cloud native technologies empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds. Containers, service meshes, microservices, immutable infrastructure, and declarative APIs exemplify this approach.

These techniques enable loosely coupled systems that are resilient, manageable, and observable. Combined with robust automation, they allow engineers to make high impact changes frequently and predictably with minimal toil.

The Cloud Native Computing Foundation seeks to drive adoption of this paradigm by fostering and sustaining an ecosystem of open source, vendor-neutral projects. We democratize state-of-the-art patterns to make these innovations accessible for everyone.

l.cncf.io

©2020 CNCF



1. CONTAINERIZATION

- Commonly done with Docker containers.
- Any code application and dependencies (even Docker itself) running in an image can be containerized.
- Container you should expect towards something suitable applications and writing future functionality as microservices.

2. CI/CD

- Select Continuous Integration/Continuous Delivery (CI/CD) tool that integrates well with your existing infrastructure (e.g. a new build or deployment tool) and has a good ecosystem (e.g. a new build or deployment tool) and has a good ecosystem (e.g. a new build or deployment tool).
- Argo is a set of Kubernetes-native tools for declarative CI/CD for your applications, workflows, and events using declarative pipelines such as Argo Pipelines, Argo CD, Argo Rollouts, and Argo Events.



3. ORCHESTRATION & APPLICATION DEFINITION

- Kubernetes is the market leading orchestration solution.
- You should expect a Cloud Native Kubernetes distribution.
- Helm is a package manager for Kubernetes.
- Helm Charts are reusable Kubernetes manifests.



4. OBSERVABILITY & ANALYSIS

- Prometheus for monitoring, Grafana for dashboarding, and Elasticsearch for logging and analysis.
- For logging, you can also use Elasticsearch, or a dedicated logging solution like Fluentd.



5. SERVICE PROXY, DISCOVERY, & MESH

- Envoy is a high performance service proxy.
- Consul is a service discovery and orchestration tool.
- This path is not required for most use cases.



6. NETWORKING, POLICY, & SECURITY

- The CNCF community has developed a set of standards for networking, policy, and security.
- These standards are being implemented by various CNCF projects.



7. DISTRIBUTED DATABASE & STORAGE

- When you need a distributed database or storage, you should expect a Cloud Native solution.
- This path is not required for most use cases.



8. STREAMING & MESSAGING

- When you need better performance than a standard database, you should expect a Cloud Native solution.
- This path is not required for most use cases.



9. CONTAINER REGISTRY & RUNTIME

- Container registries store images and are used to deploy containers.
- This path is not required for most use cases.



10. SOFTWARE DISTRIBUTION

- This path is not required for most use cases.



学习路径 6-10

- **网络**

- Calico、Flannel以及Weave Net等软件用于提供更灵活的网络功能。

- **分布式数据库和存储**

- 提供更好的弹性和伸缩性能。

- **流和消息处理**

- gRPC是一个通用的RPC（远程调用）框架（类似各种框架中的RPC调用），
- NATS是一个发布/订阅和负载均衡的消息队列系统。

- **容器镜像库和运行环境**

- Harbor是目前最受欢迎的容器镜像库。

- **软件发布**

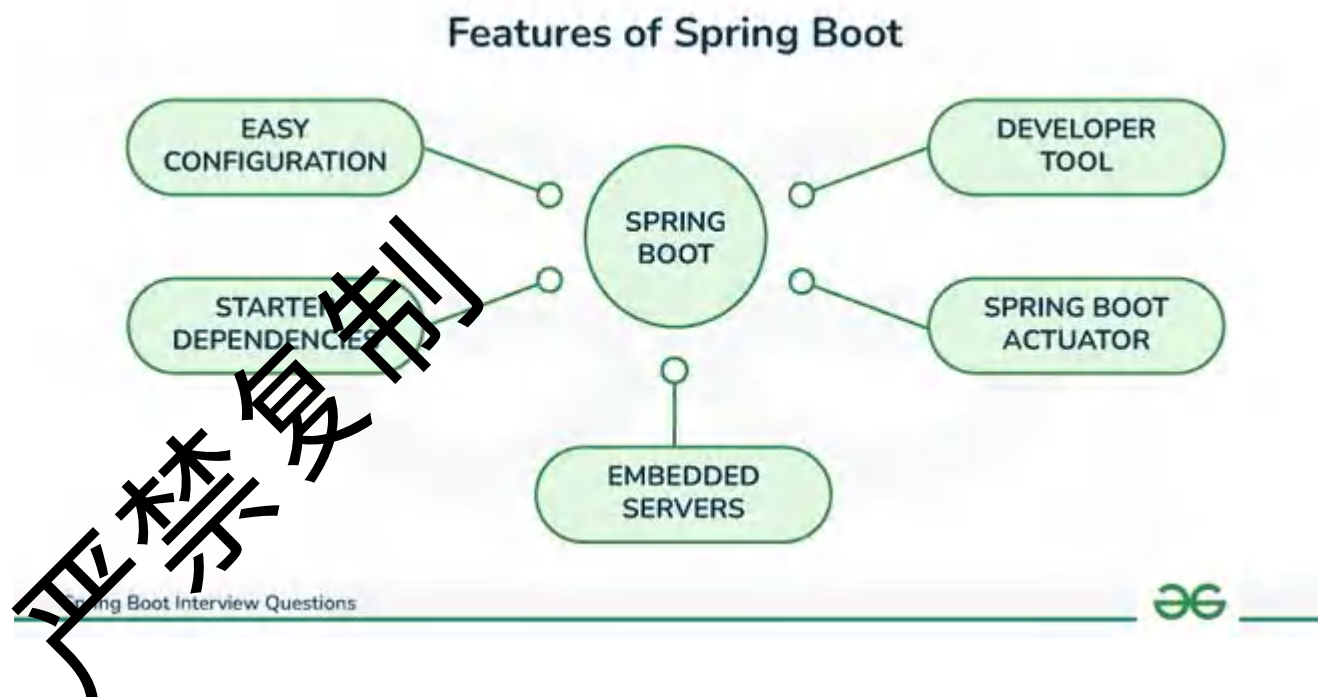
- 最后可以借助Notary等软件用于软件的安全发布。



02 复制 快速开发-spring boot

问题1：如何快速开发微服务

- Spring boot
 - 启动依赖（集成parent）
 - 内嵌的server
 - 自动配置推理
 - Actuator



启动依赖：传统的java命令

- 最原始的方式：
 - `java -cp "classes:lib/dependency1.jar:lib/dependency2.jar" com.example.MyMainClass`
- 问题：
 - jar包是错误的
 - jar包版本升级
 - jar包冲突

禁止复制

```
Exception in thread "main" java.lang.NoSuchMethodError:
'com.fasterxml.jackson.databind.ObjectMapper.readTree(Ljava/io/InputStream;Z)Lcom/fas
terxml/jackson/databind/JsonNode;'
at com.example.some_library.DataProcessor.processJsonStream(DataProcessor.java:75)
at com.mycompany.myapp.MainApplication.loadAndProcessData(MainApplication.java:42)
at com.mycompany.myapp.MainApplication.main(MainApplication.java:15)
```

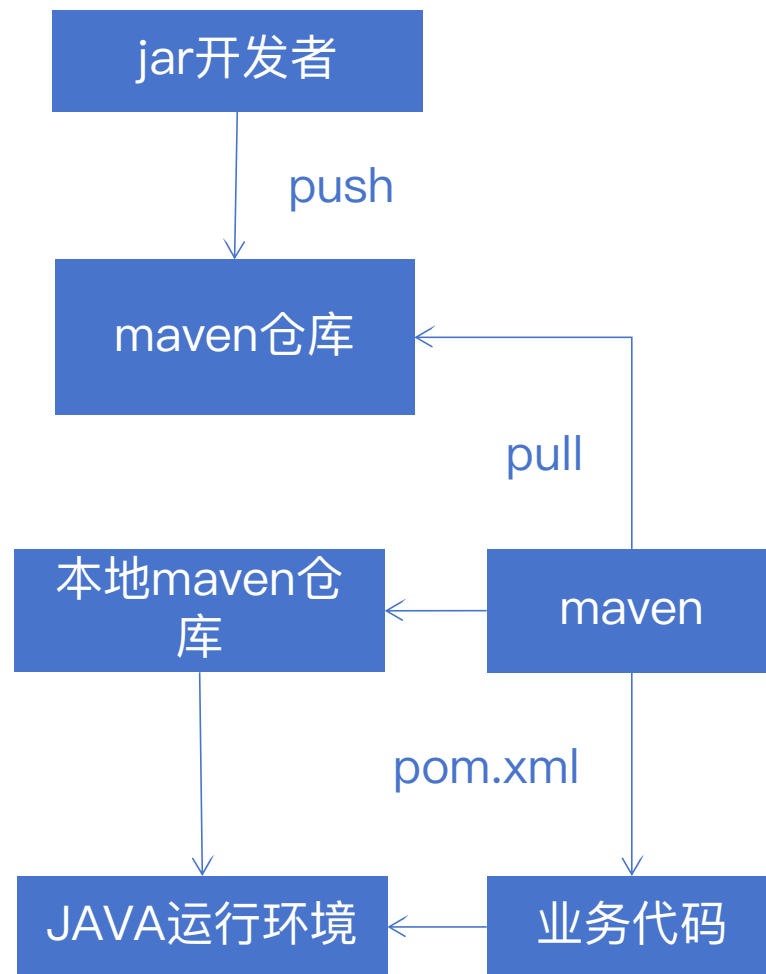
启动依赖：Maven

- Maven依赖：

```
<dependency>
  <groupId>org.springframework.data</groupId>
  <artifactId>spring-data-bom</artifactId>
  <version>${spring-data-bom.version}</version>
</dependency>
```

- 解决版本冲突：

- 显示声明
- 排除：
 - depend on A 1.0, exclude C, depend on C 2.0
- dependencyManagement



启动依赖：Spring Boot

- 统一和集中的依赖版本管理
 - Spring Boot 团队精心挑选和测试过的、相互兼容的各种第三方库的版本号。
- 精心策划的“意见化”依赖集合
 - 每个 Spring Boot Starter（例如 `spring-boot-starter-web`、`spring-boot-starter-data-jpa`）都代表了一个特定功能的“意见化”依赖集合。
- 减少传递性依赖的复杂性
 - 所有的 Starter 都与 Spring Boot 的主版本号保持一致。

禁止复制

Spring Boot自动化配置

- @Configuration一种配置方式
 - xml配置
 - @Component, @Autowire.....
- @EnableAutoConfiguration
 - spring.factories
- @Conditional
 - @ConditionalOnClass或MissingClass
 - @ConditionalOnBean 或MissingBean
 - @ConditionalOnProperty
- 补充对象
 - Web 应用, 数据库, 消息队列, 缓存, 日志.....

```
// AppConfig.java
@Configuration // 标记这是一个配置类
public class AppConfig {

    @Bean // 标记这个方法返回一个 bean
    public MyService myService() {
        // 在这里可以进行复杂的初始化逻辑
        return new MyServiceImpl(myDao());
    }

    @Bean
    public MyDao myDao() {
        return new MyDaoImpl();
    }
}
```

Spring Boot自动化配置过程

- 启动： Spring Boot 应用启动， @EnableAutoConfiguration 生效。
- 扫描： META-INF/spring.factories 列出的自动化配置类。
- 评估： 对于每个自动化配置类，评估 @ConditionalOn... 注解。
- 激活： 只有当所有条件都满足时，该自动化配置类才会被激活。
- 配置： 被激活的自动化配置类会向 Spring 容器中注册相应的 Bean，从而完成“缺失信息”的自动补充。

```
@SpringBootApplication
public class MyAppApplication implements CommandLineRunner {

    @Autowired // 自动注入由自动配置创建的 GreetingService
    private GreetingService greetingService;

    @Override
    public void run(String... args) throws Exception {
        System.out.println( greetingService.greet("World"));
    }
}
```

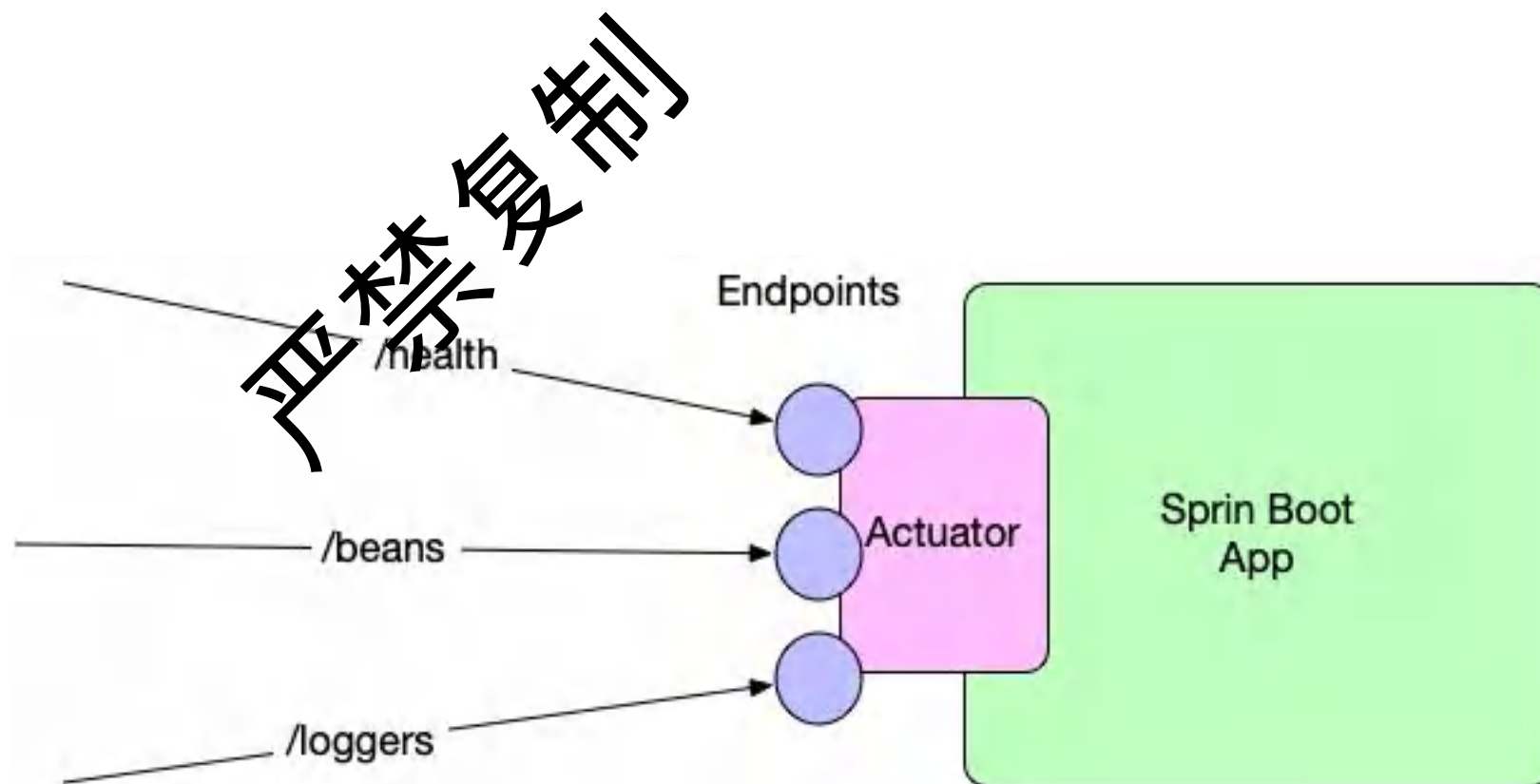
Spring actuator

- 提供app的内部信息:

- /actuator/health
- /actuator/info
- /actuator/metrics
- /actuator/env
- /actuator/beans

- 安全性

- 集成 Spring Security
- 网络隔离
- 仅暴露必要的端点

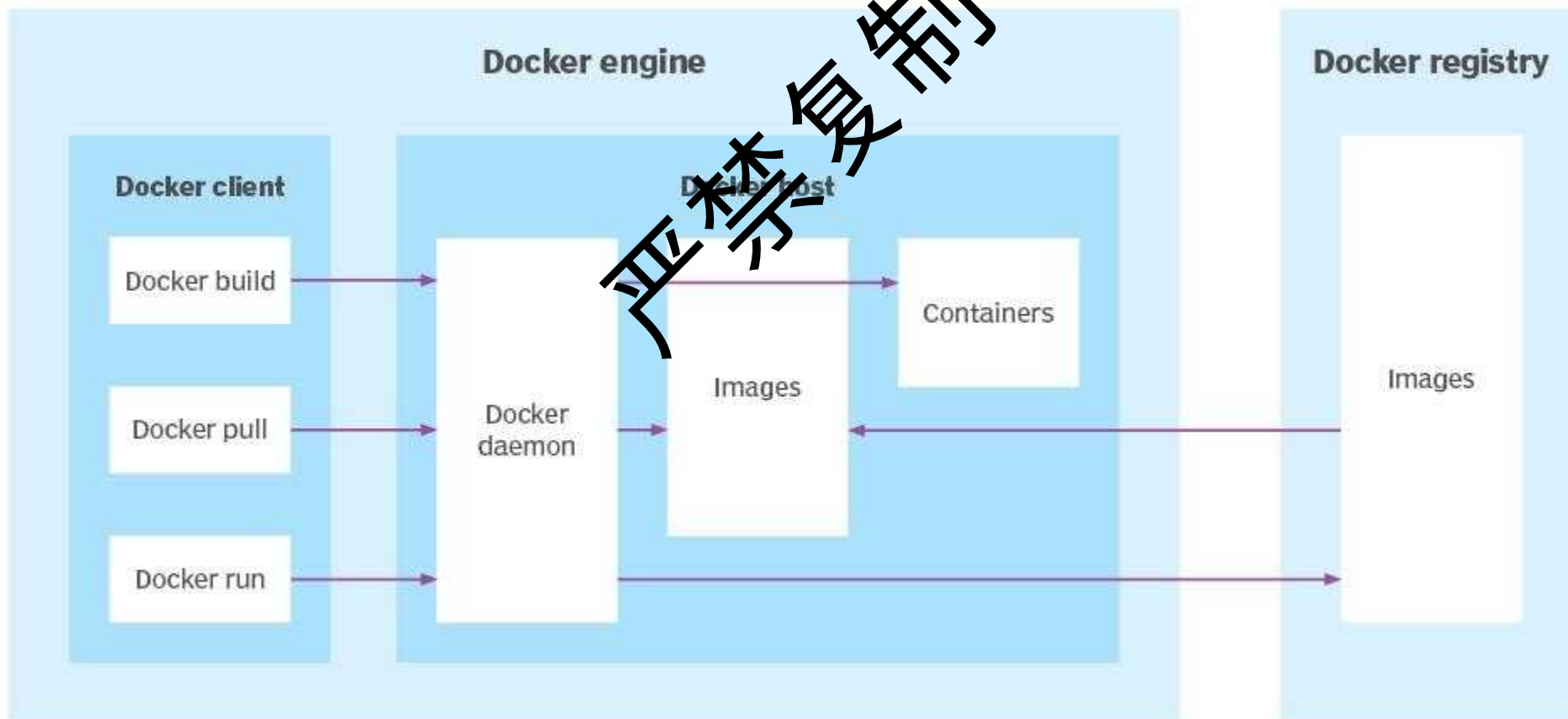




03 复制控制 容器化部署-docker

2 Docker

- 允许开发者将应用程序及其所有依赖项打包到一个被称为“容器”的标准化、轻量级、可移植的单元中，从而确保应用程序在任何环境中都能以相同的方式运行。
- 线上docker: <https://labs.iximiuz.com/tutorials/docker-container-management-commands>



Docker的组件

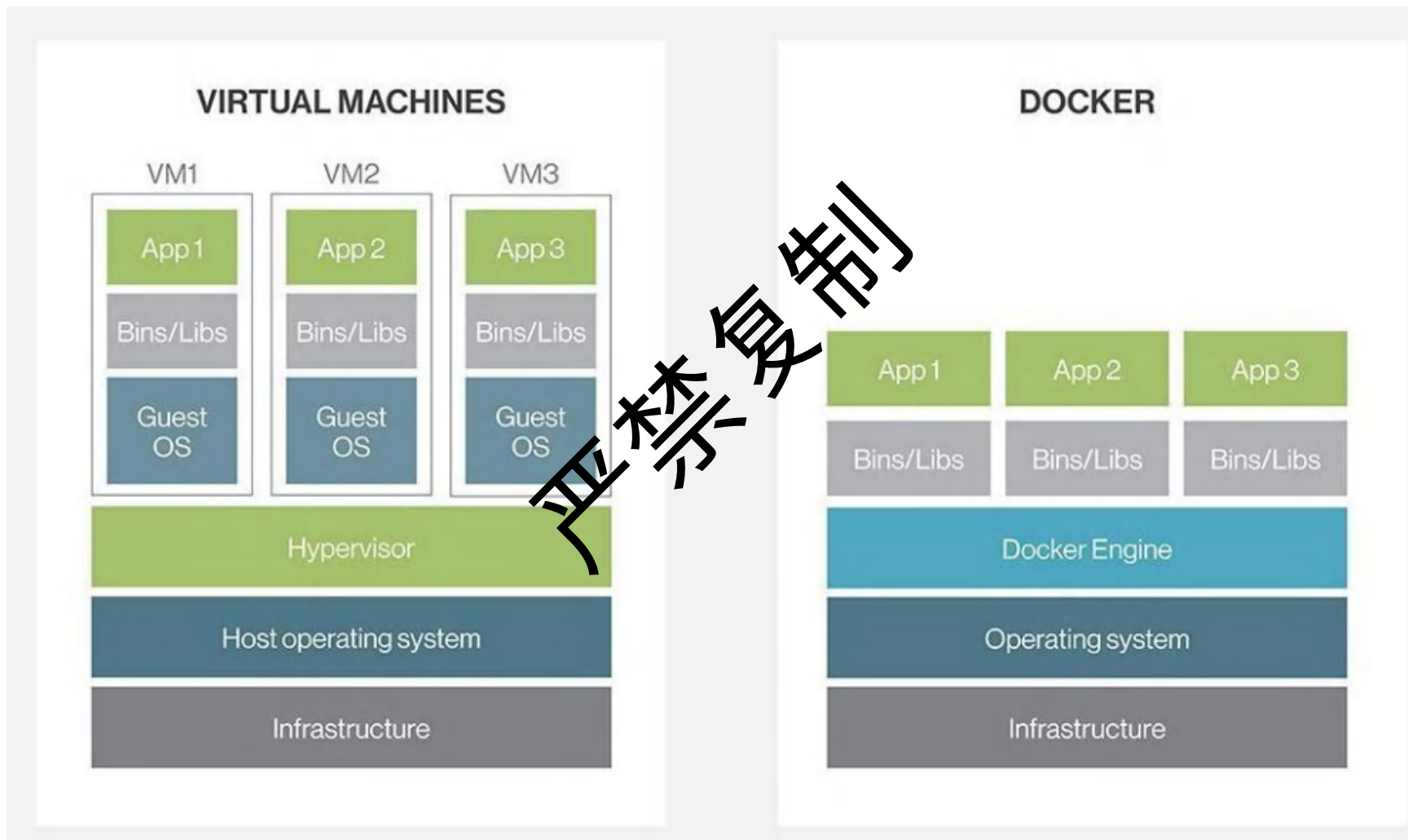
- Docker Engine：
 - 运行和管理容器的核心服务。
- Docker Images（镜像）：
 - 容器的“蓝图”或“模板”。它是只读的，包含了应用程序和所有依赖的静态快照。
- Docker Containers（容器）：
 - 镜像的运行实例。它是可读写的，是应用程序实际运行的地方。
- Dockerfile：
 - 用于自动化构建 Docker 镜像的文本文件，其中包含了一系列指令。
- Docker Registry（注册中心）：
 - 用于存储和分享 Docker 镜像的地方，最常用的是 Docker Hub

Docker Image

- 基础操作系统（Base OS）组件：
 - 精简的 Linux 发行版
- 运行时环境（Runtime Environment）：
 - 应用程序所需的语言运行时或虚拟机，如JVM
- 应用程序代码和相关资源：
 - 编译后的二进制文件或脚本
- 应用程序依赖库：
 - 所依赖的第三方库或框架
- 环境变量和配置文件：
 - 配置参数，数据库连接字符串、API 密钥、日志级别

禁止转载

Docker ≠ VM



CGroup and Namespace

- namespaces 它提供了容器的“隔离性”：
 - 提供独立的运行环境：每个容器都有自己独立的进程空间、网络配置、文件系统视图等。
 - 避免冲突：不同容器之间不会因为端口占用、文件路径冲突、进程 ID 重复等。
 - 增强安全性：即使容器内部的进程获得了 root 权限，由于 user 命名空间的存在。
- cgroups 主要提供以下功能：
 - 资源限制 (Resource Limiting)：限制一个容器最多使用 512MB 内存或 2 个 CPU 核心。
 - 优先级 (Prioritization)：让一个容器在 CPU 竞争时获得更高的优先级。
 - 审计 (Accounting)：计量进程组实际使用的资源量，例如CPU 时间或内存。
 - 控制 (Control)：冻结、恢复或终止一个进程组中的所有进程。

Notary: Docker守卫者



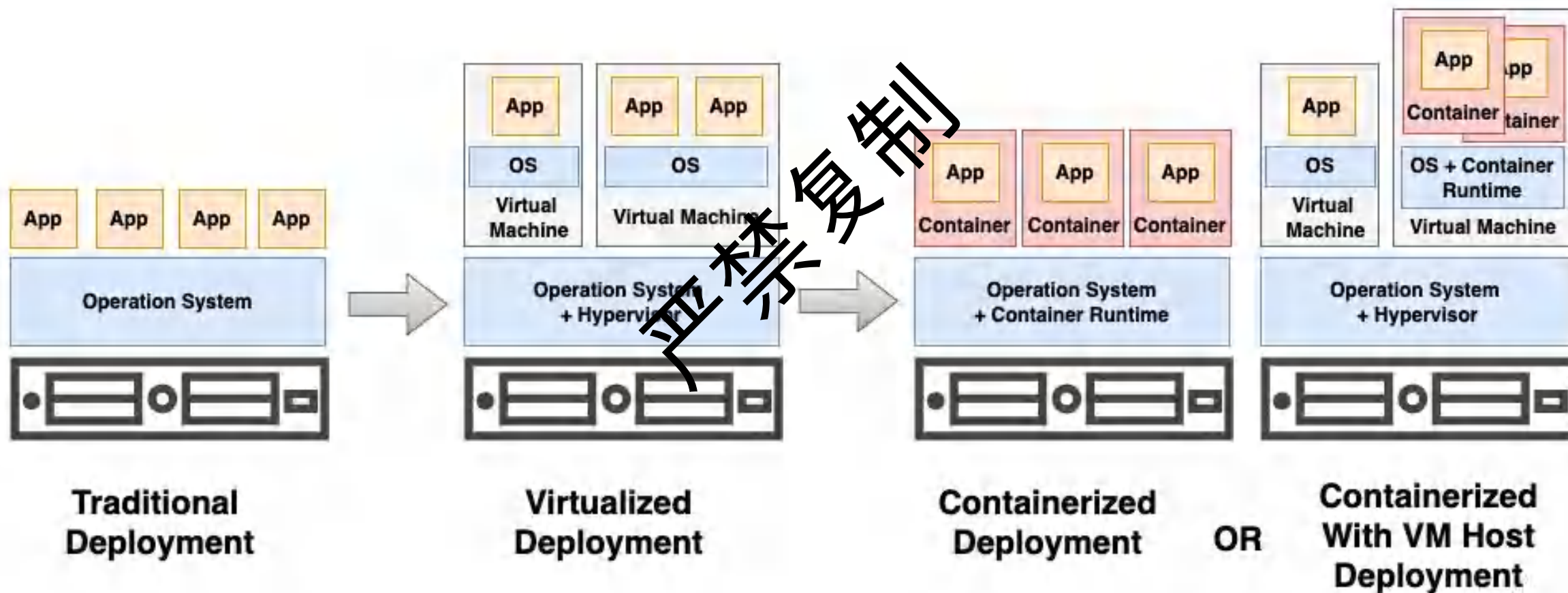
- 核心功能：
 - 允许发布者对数据进行数字签名，
 - 并允许消费者验证这些签名的真实性。
- 基于 TUF
 - 多层密钥、密钥轮换、离线密钥、回滚保护等机制，来抵御各种攻击
- 解决的问题：
 - 中间人攻击 (Man-in-the-Middle Attacks): 确保内容在传输过程中没有被拦截和篡改。
 - 镜像篡改 (Image Tampering): 防止恶意方修改容器镜像或配置。
 - 供应链攻击 (Supply Chain Attacks): 确保你运行的软件组件是来自你信任的来源。
 - 回滚攻击 (Rollback Attacks): 防止攻击者强迫你使用旧的、有漏洞的版本。

禁止复制



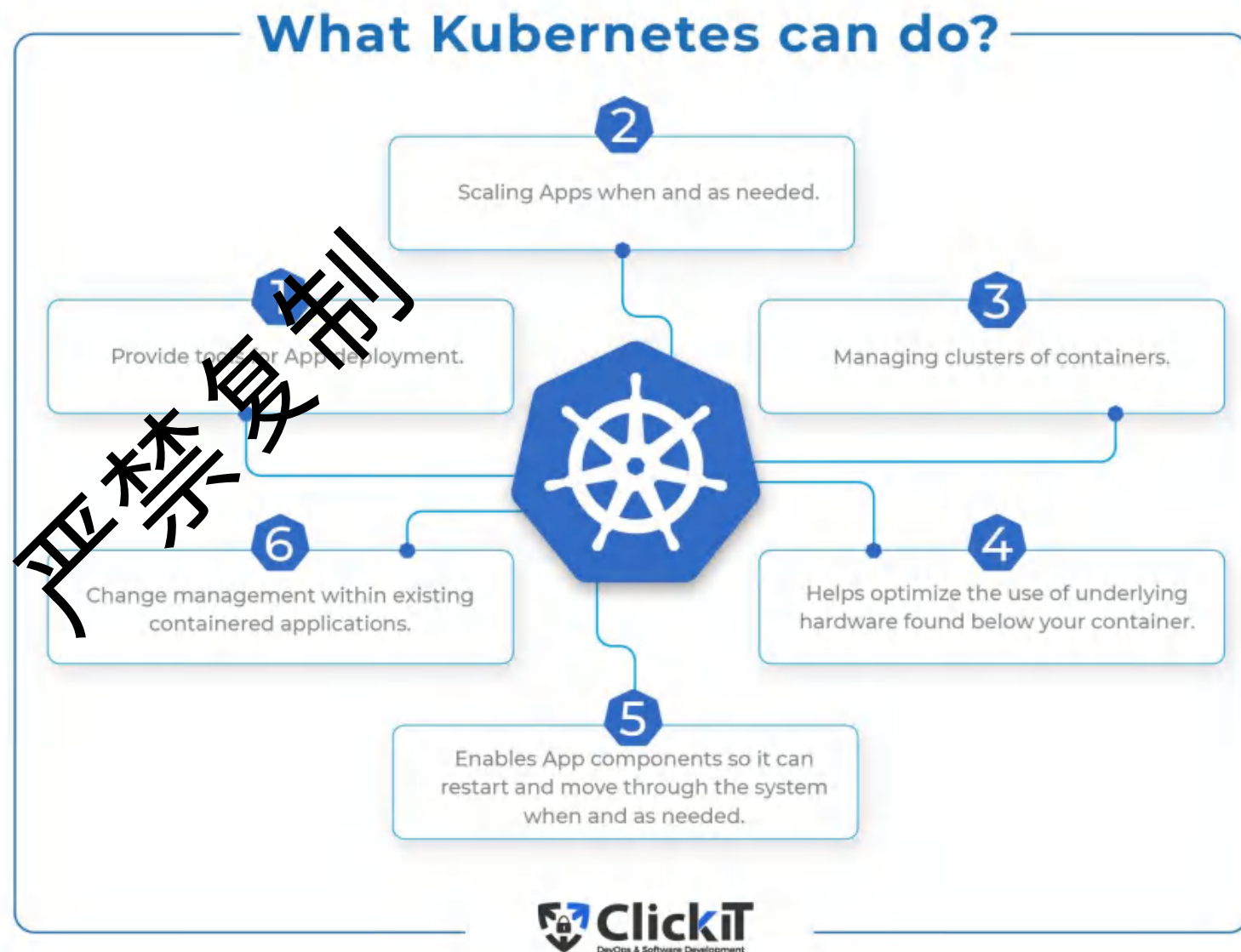
04 复制 微服务编排

Kubernetes: 微服务的部署方式



K8S的5个特性

- 支持APP部署
- 按需伸缩
- 管理容器集群
- 优化底层的硬件
- 重启或者迁移
- 变更或者升级



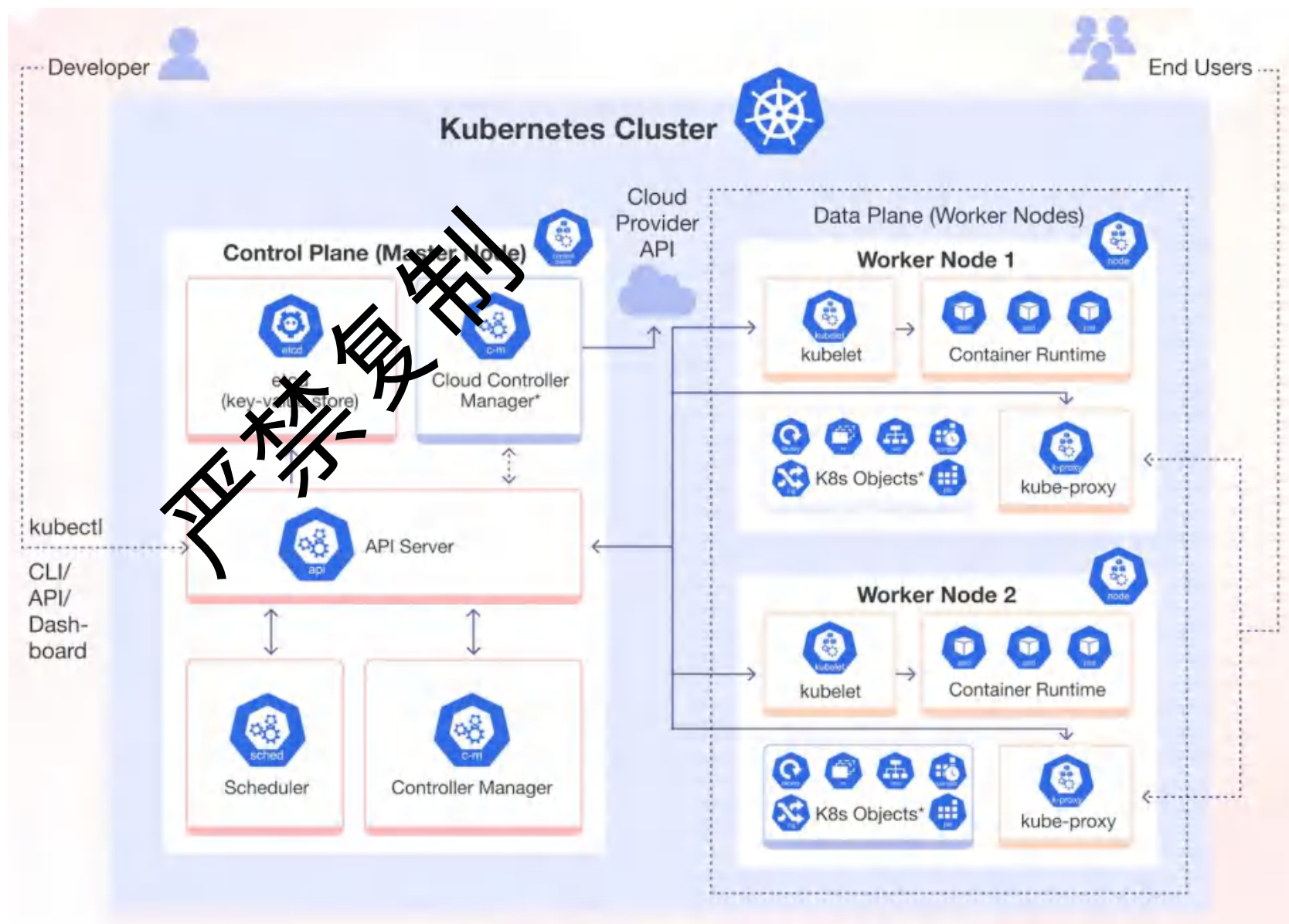
K8S架构

控制组件

1. kube-apiserver (API 服务器)
2. etcd (分布式键值存储)
3. kube-scheduler (调度器)
4. kube-controller-manager (控制器管理器)
5. cloud-controller-manager (云控制器管理器)

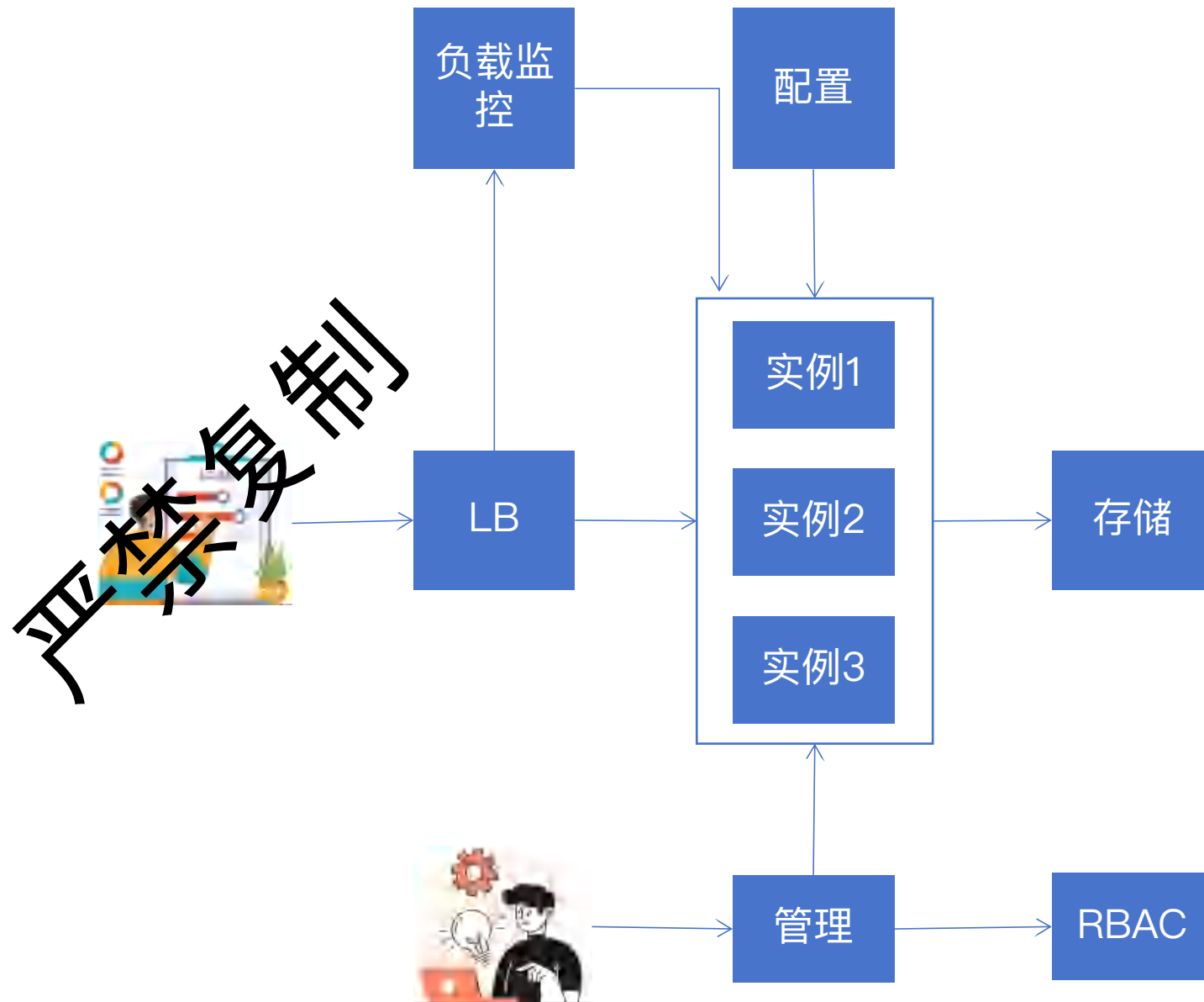
节点组件

1. kubelet
2. kube-proxy (网络代理)
3. Container Runtime (容器运行时)



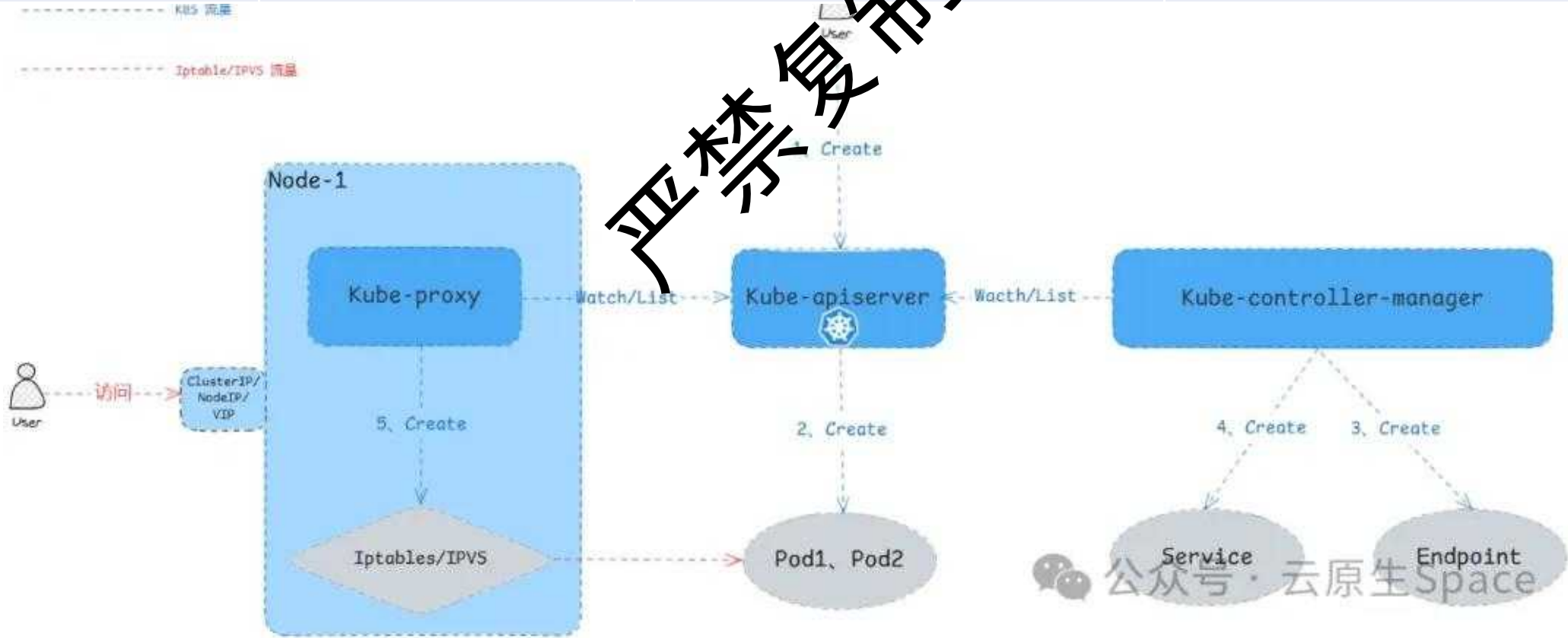
K8S的管理功能

1. 工作负载
2. 服务发现与负载均衡
3. 存储
4. 配置
5. 安全与访问控制
6. 集群状态与基础设施



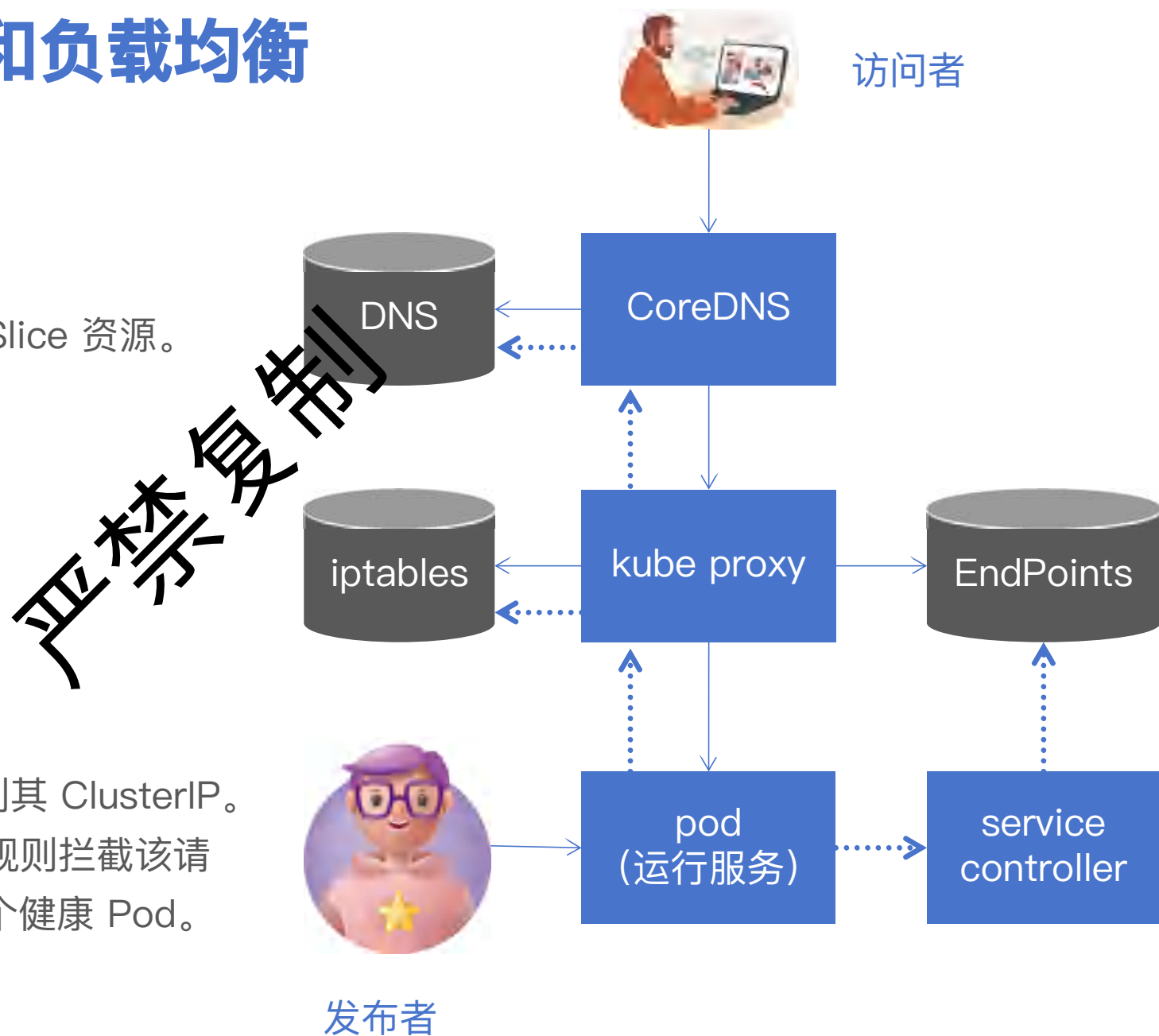
K8S上的service访问几种类型

类型	访问范围	访问方式	典型场景
ClusterIP	集群内部	通过 ClusterIP	服务间通信
NodePort	集群外部	NodeIP:NodePort	简单公网暴露
LoadBalancer	集群外部	云负载均衡器分配的IP/域名	生产环境公网服务
ExternalName	集群外部	DNS 解析为外部服务域名	访问外部服务



K8S的核心：服务发现和负载均衡

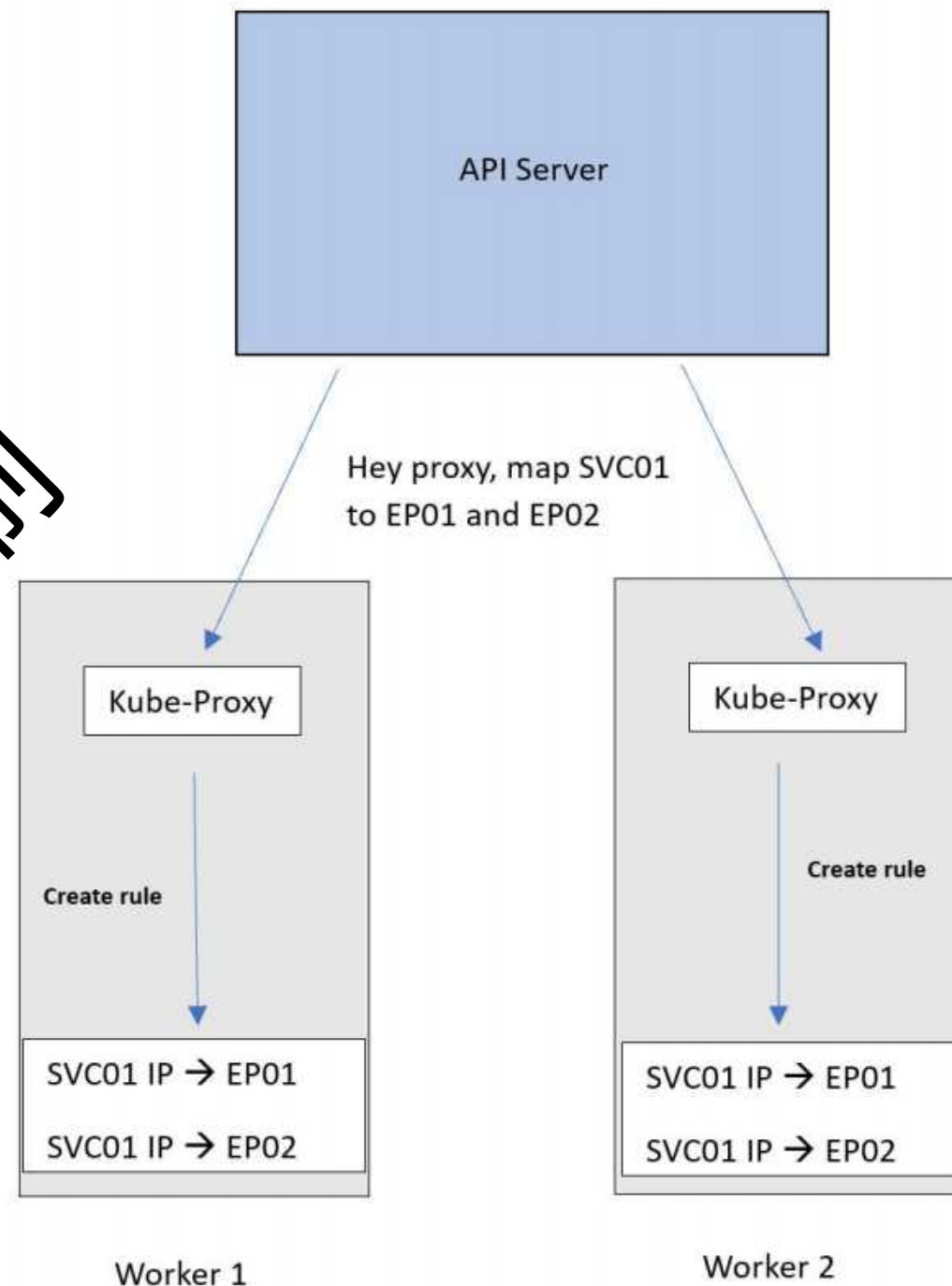
1. 创建 Pod 和 Service。
2. Service Controller 发现 Pod：
 1. 创建/更新 Endpoints/EndpointSlice 资源。
3. CoreDNS 注册 DNS：
 1. 为Service创建 DNS 记录。
4. kube-proxy：
 1. 配置 iptables 或 IPVS 规则。
5. 客户端 Pod 发起请求：
 1. 通过 Service 的 DNS 名称解析到其 ClusterIP。
 2. 节点的 kube-proxy 配置的网络规则拦截该请求，并将其负载均衡到后端的一个健康 Pod。



K8S负载均衡：Kube-proxy

- 主要职责：
 - 为 Kubernetes Service 提供四层（L4，TCP/UDP）负载均衡和流量转发。
- 工作方式：
 - 通过 userspace、iptables 或 ipvs 模式，在节点层转发流量到后端 Pod。
- 特点：
 - 实现简单，主要负责集群内部 Service 的流量分发和负载均衡。

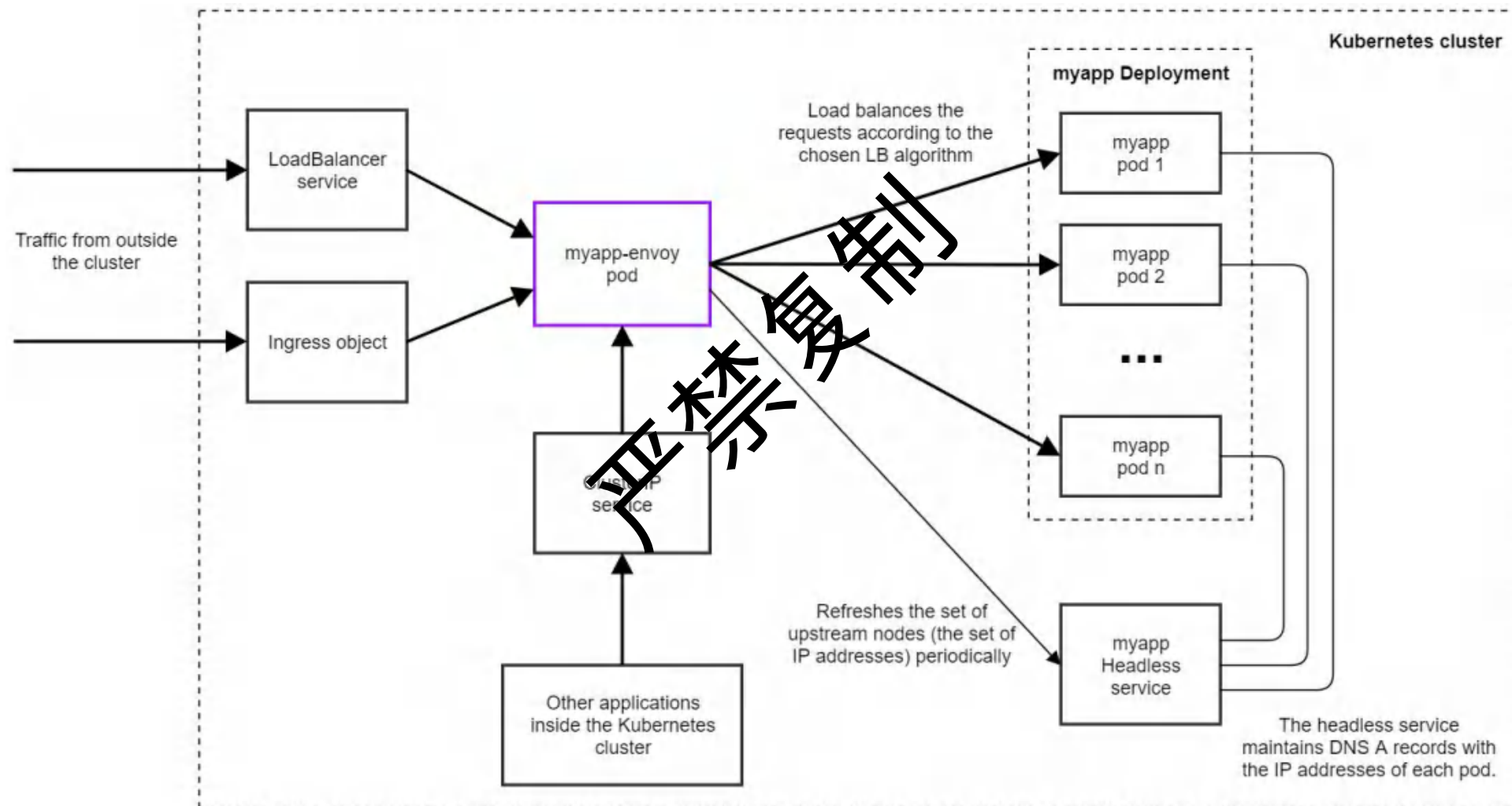
严禁复制



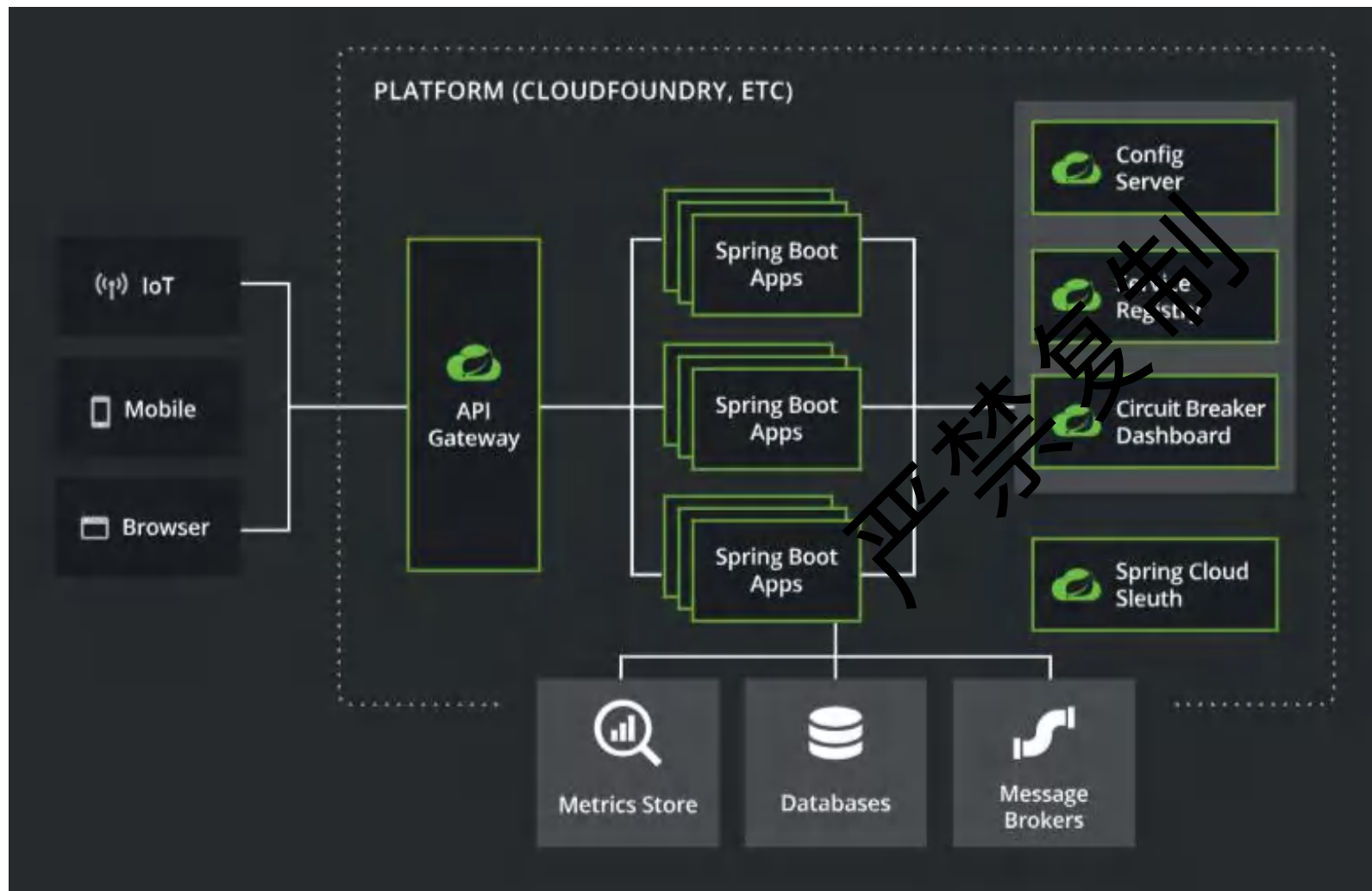
K8S 负载均衡 Envoy

- 高级负载均衡 (Advanced Load Balancing)
 - 轮询、最少连接、随机、一致性哈希
 - L4 (TCP) 和 L7 (HTTP/gRPC) 负载均衡
- 丰富的流量管理 (Rich Traffic Management):
 - 路由 (Routing): 根据请求的路径、头部、Host 等信息将流量路由到不同的上游服务或版本 (例如 A/B 测试、金丝雀发布)
 - 重试 (Retries), 超时 (Timeouts), 熔断 (Circuit Breaking), 限流 (Rate Limiting)
 - 流量镜像 (Traffic Mirroring/Shadowing)
 - 故障注入 (Fault Injection):
- 其它特性
 - 强大的可观察性 (Powerful Observability)
 - 安全 (Security):
 - 协议支持 (Protocol Support):
 - 可扩展性 (Extensibility):

K8S的负载均衡: Envoy



Spring Cloud: 运行在K8S之外



解决微服务集成的问题。

外部服务：

1. API网关
2. 服务发现
3. 集中化配置管理

内部组件：

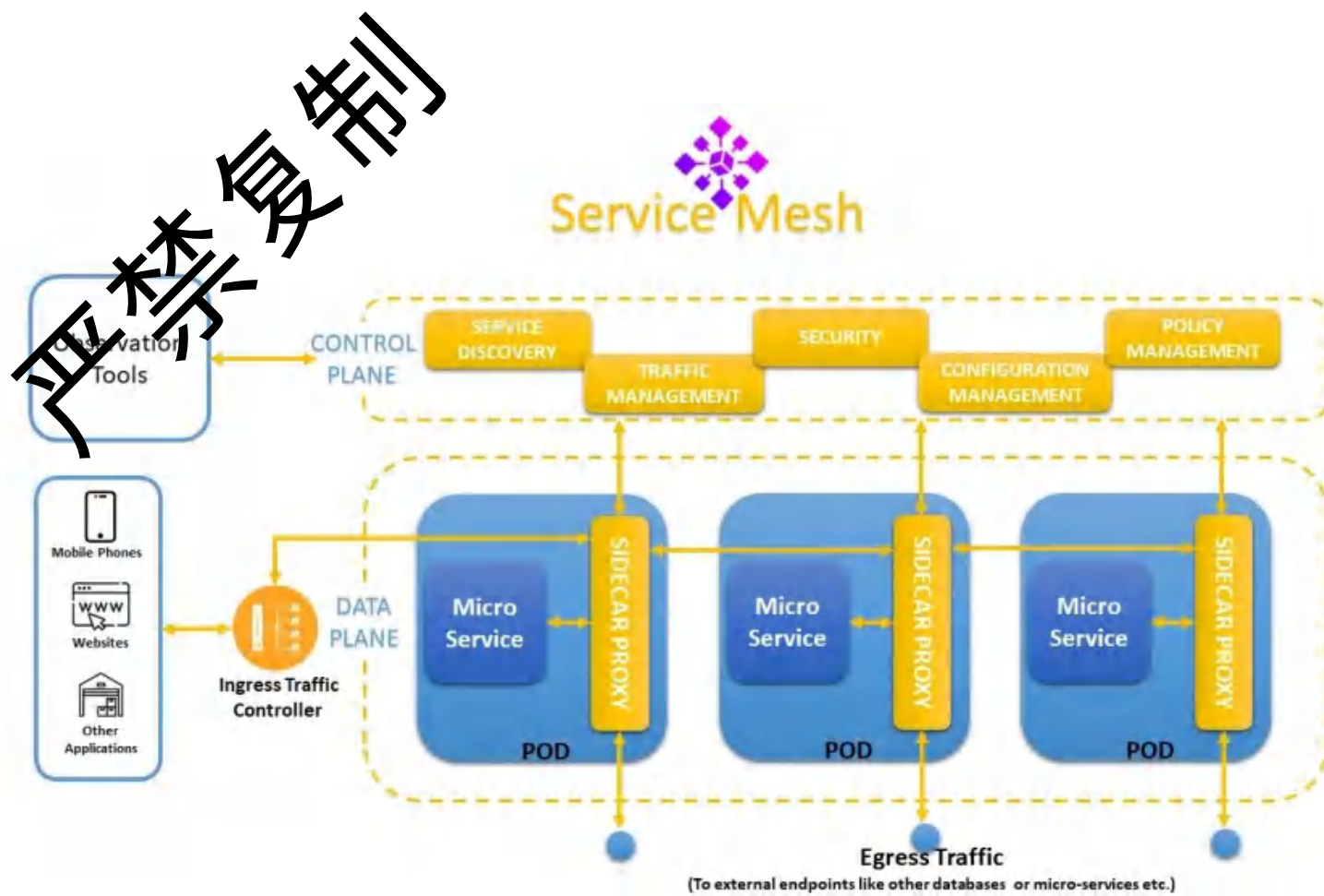
1. 客户端负载均衡
2. 断路器
3. 分布式链路跟踪
4. 消息驱动

Spring Cloud的服务发现 VS K8S的服务发现

特性	Spring Cloud 服务发现 (e.g., Eureka)	Kubernetes 服务发现
层面	应用层 / 客户端层	平台层 / 基础设施层
注册方式	应用程序 主动注册 到独立的注册中心	K8s 平台 自动感知 Pods 和 Services 的创建与销毁
发现方式	客户端 拉取 实例列表, 客户端负载均衡 (Ribbon/LoadBalancer)	DNS 解析到 Service Cluster IP, Kube-proxy 负载均衡 (L4)
负载均衡	客户端侧 (应用代码中)	K8s 平台侧 (Kube-proxy)
健康检查	客户端心跳 + 注册中心剔除	K8s Liveness/Readiness Probes
核心组件	独立的注册中心服务 (如 Eureka Server)	K8s 集群本身 (API Server, Kube-proxy, CoreDNS)
暴露信息	服务实例的 IP:Port	Service 的 Cluster IP :Port (抽象后端 Pods)
语言相关性	客户端库通常是特定语言/框架的 (如 Java/Spring)	完全语言和框架无关
运维成本	需要额外部署和维护注册中心	K8s 内置, 无需额外运维独立组件

Service Mesh服务网格：对K8S的补充

- 流量管理：
 - 更细粒度的路由（A/B 测试、金丝雀发布）、流量整形、故障注入、重试、超时、熔断、限流等。
- 可观察性：
 - 统一的指标收集、分布式追踪、日志聚合。
- 安全性：
 - 服务身份认证、双向 TLS (mTLS) 加密通信、细粒度的授权策略。
- 平台无关性：
 - 将这些通信逻辑从应用代码中剥离，使得应用代码可以专注于业务逻辑，不依赖特定语言或框架。



Service Mesh和K8s的关系

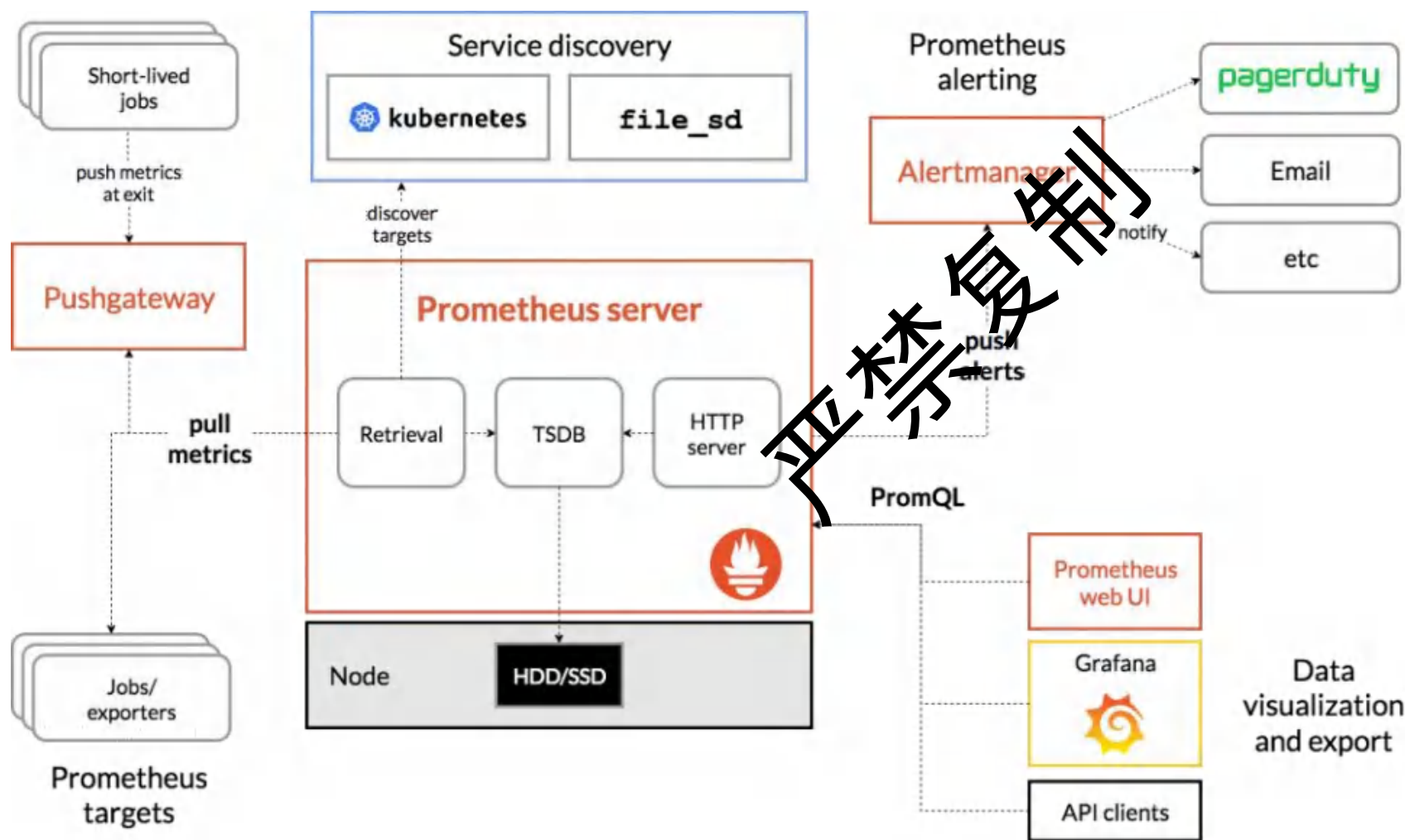
- Kubernetes 城市基础设施：
 - 道路（网络）、电力（计算资源）、供水（存储）、楼房（Pod），
 - 微服务=居民。
- Service Mesh 交通管理+安保系统：
 - 智能交通灯、摄像头、GPS 导航、安检关卡等，使得车辆（服务间的请求）可以更高效、更安全、更可控地在城市中穿梭，
 - 追踪每一辆车的行程。
- 使用：
 - 先在 Kubernetes 上部署你的微服务，
 - 引入 Service Mesh（例如 Istio、Linkerd 等）来管理这些微服务之间的通信。





05 复制 日志和监控

Prometheus（普罗米修斯）架构



Prometheus 是一个开源的监控和报警工具包：

1. 发现服务；
2. 拉取指标；
3. 发出报警；
4. 图形展示；

Prometheus拉取哪些数据

- 数据：

- 操作系统： CPU 使用率、内存占用、磁盘 I/O、网络流量等。
- 应用程序： HTTP 请求总数、错误率、API 响应时间、数据库连接池大小等。
- 数据库： 查询 QPS、慢查询数量、连接数、缓存命中率等。
- 消息队列： 消息堆积量、生产者/消费者数量等。
- 硬件设备： 温度、风扇转速等。

- 格式：

- 数据内容不一样，但数据的暴露格式是统一的。
- `metric_name{label1="value1", label2="value2"} value [timestamp]`

如何获得数据

- 应用程序集成 Client:
- 各种 Exporter (数据导出器):
 - 数据库、消息队列、操作系统、硬件设备
- 云原生组件内置Prometheus 格式:
 - cAdvisor: 容器的资源使用
 - CPU、内存、网络
 - kube-state-metrics:
 - (如 Pod、Deployment、Service、Node 等)。
 - CoreDNS:

```
// 2. 定义 Gauge (仪表盘)  
// 用于显示当前正在处理的请求数  
private static final Gauge IN_PROGRESS_REQUESTS = Gauge.build()  
    .name("my_app_in_progress_requests")  
    .help("Number of requests currently being processed.")  
    .register();
```

```
// 1. Gauge: 请求开始时, 增加正在处理的请求数  
IN_PROGRESS_REQUESTS.inc();
```

```
// 3. Counter: 请求处理完成时, 增加总请求数  
REQUEST_TOTAL.labels(method, path).inc();
```

Prometheus统一数据格式

metric_name{label1="value1", label2="value2"} value [timestamp]

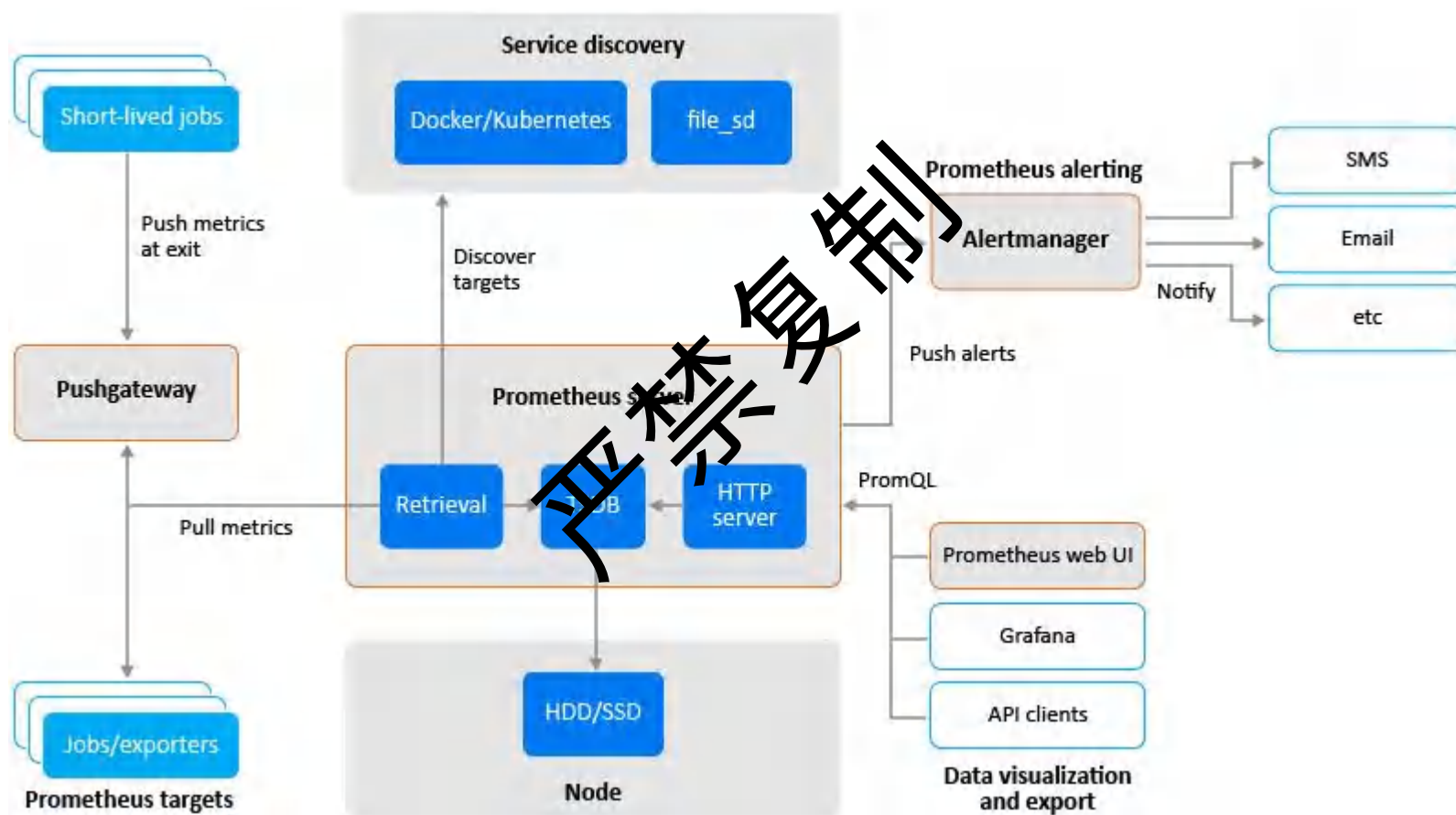
Counter (计数器):一个单调递增的累积计数器	Gauge (计量器)任意上下浮动的值
<pre># TYPE http_requests_total counter http_req_total{method="post",path="/api/users"} 100 http_requests_total{method="get",path="/"} 250</pre>	<pre># TYPE temperature_celsius gauge temperature_celsius{location="server_room"} 25.5</pre>
Histogram (直方图)将其聚合到可配置的桶中。	Summary (摘要):在客户端计算和暴露分位数
<pre>http_request_duration_seconds_bucket{le="0.1"} 100 http_request_duration_seconds_bucket{le="0.5"} 150 http_request_duration_seconds_bucket{le="1"} 180 http_request_duration_seconds_bucket{le="+Inf"} 200 http_request_duration_seconds_sum 50.0 http_request_duration_seconds_count 200</pre>	<pre># rpc_duration_seconds RPC latency in seconds. # TYPE rpc_duration_seconds summary rpc_duration_seconds{quantile="0.5"} 0.01 rpc_duration_seconds{quantile="0.9"} 0.02 rpc_duration_seconds{quantile="0.99"} 0.03 rpc_duration_seconds_sum 1000 rpc_duration_seconds_count 10000</pre>

Prometheus的数据查询

```
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

字段	含义
node_cpu_seconds_total:	Node Exporter 提供的 CPU 时间总和。
{mode="idle"}:	CPU 处于空闲模式的时间。
[5m]:	表示在过去 5 分钟的时间窗口。
rate(...):	计算指定时间窗口内计数器。
avg by (instance):	如果你有多个 Node Exporter 实例，这会按实例进行平均。
* 100:	将比例转换为百分比。
100 - (...):	从 100 中减去空闲 CPU 百分比，从而得到利用率。

Prometheus 数据的应用



Grafana的图形展示



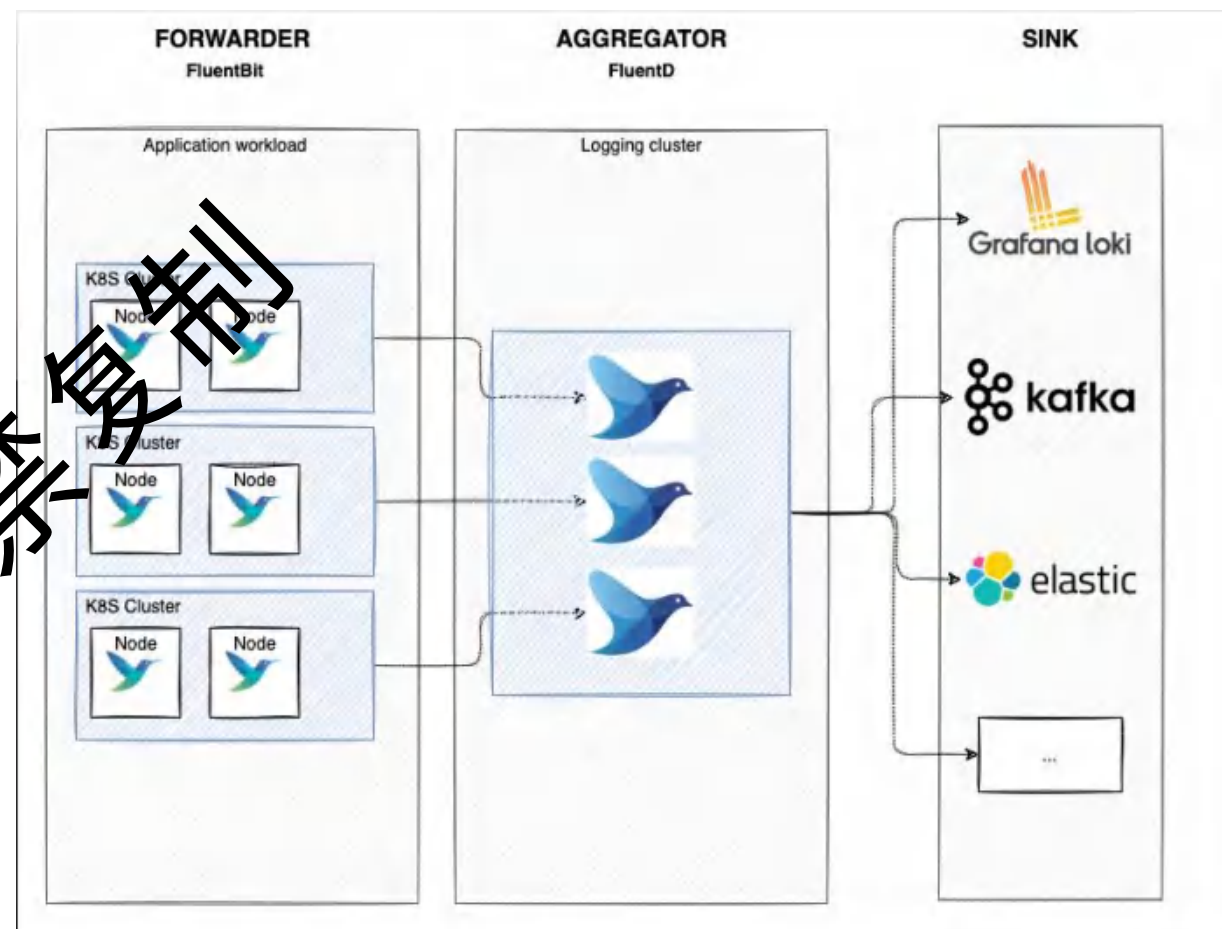
Grafana的特点

- 强大的数据可视化能力
- 广泛的数据源支持
- 告警功能
- 开源和社区驱动
- 易用性和美观的 UI
- 可扩展性和集成性
- 日志和追踪集成 (Loki & Tempo & Mimir & Pyroscope)

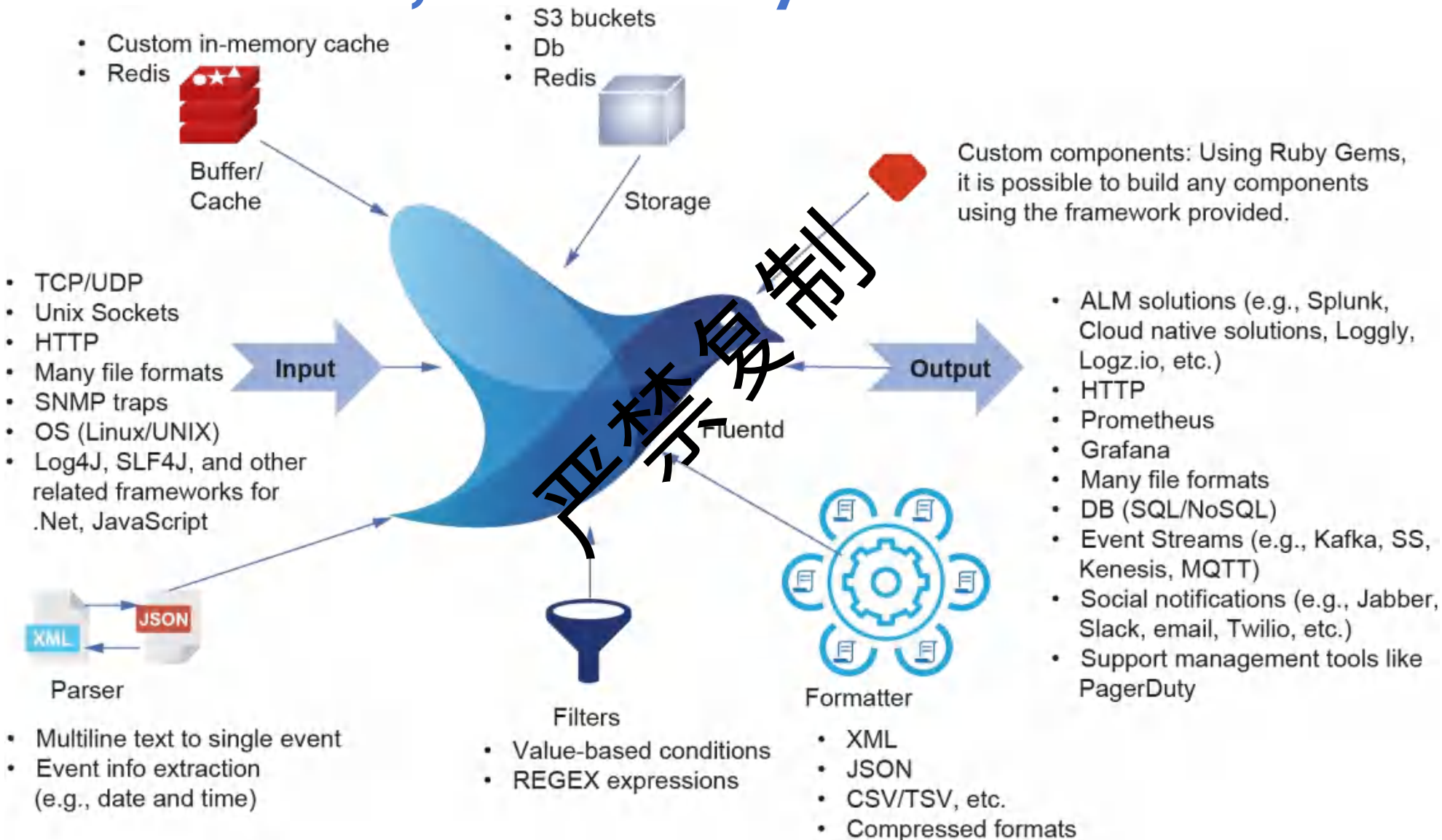
禁止复制

Fluent 解释型日志

- Fluent Bit:
 - 每个节点上部署 Fluent Bit 作为 DaemonSet,
 - 负责收集容器和节点日志, 转发到集群外部的 Fluentd 实例。
- Fluentd 实例则作为中心聚合器,
 - 接收来自 Fluent Bit 实例的日志;
 - 进行进一步的过滤、转换、聚合, 并最终将其发送到持久化存储。



Prometheus=what, Fluent=why



Fluent Bit Plugins

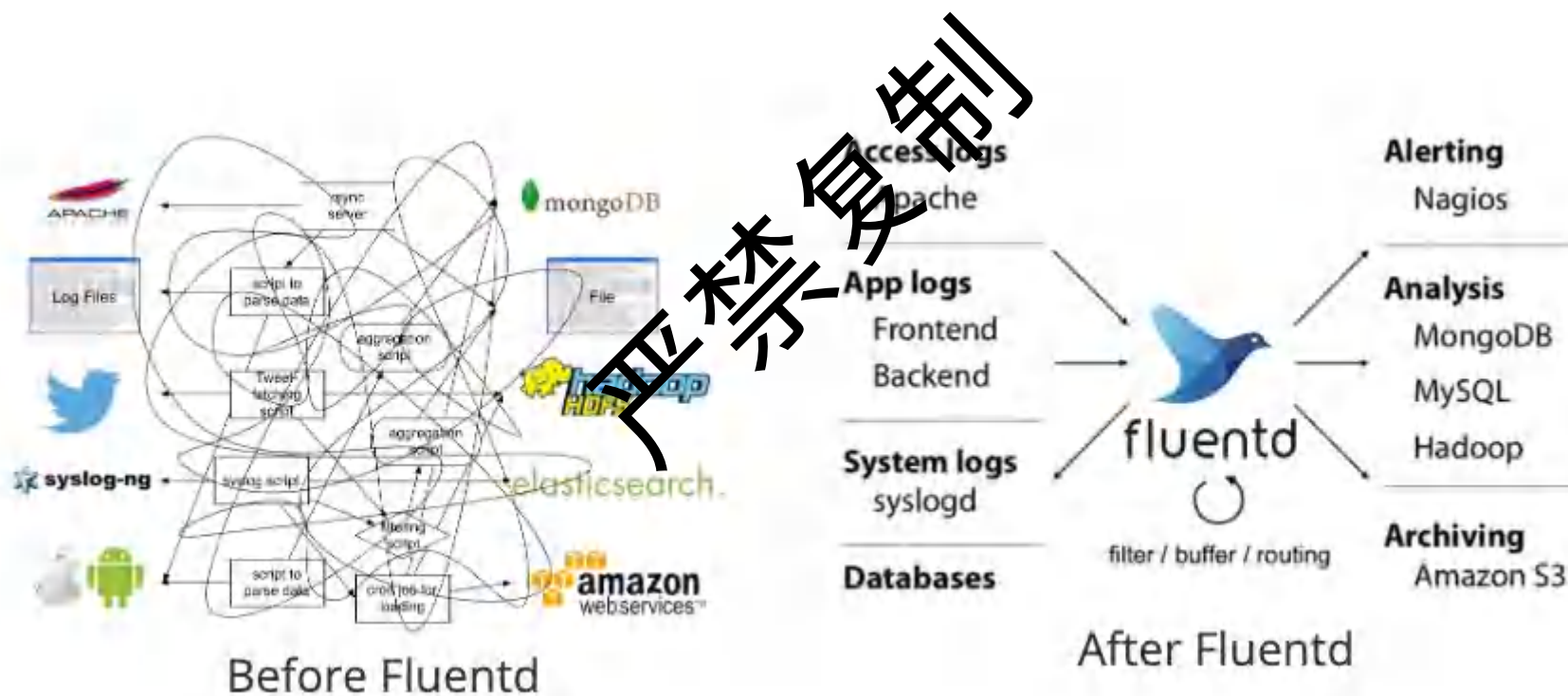
- 输入插件 (Input Plugins / Sources)
 - 负责从各种数据源收集日志。
 - tail, http, syslog, forward, tcp/udp, exec, kubernetes_metadata
- 解析器 (Parsers):
 - 输入插件或过滤插件的一部分，负责将原始的文本日志解析成结构化的数据
 - json, regexp, apache, nginx, logfmt, itsv
- 过滤插件 (Filter Plugins):
 - 在日志从输入到输出的过程中，对日志数据进行转换、修改、过滤或丰富。
 - grep, record_transformer, mutate, geoip, kubernetes_metadata, throttle
- 输出插件 (Output Plugins / Destinations):
 - stdout, file, elasticsearch, kafka, s3, splunkhec, http, mongodb,
- 缓冲插件 (Buffer Plugins):
 - 临时存储日志数据，以应对网络中断、目标系统过载或批处理需求

Fluent Bit的数据模型

- Fluent 内部处理的所有日志数据都遵循一个统一的结构：
 - Tag (标签), Time (时间戳), Record (记录)。
- Tag (标签):
 - 一个字符串, 用于标识日志的来源或类型, 是 Fluent 路由数据流的关键。
 - 例如: `app.web.access`, `system.auth.log`
- Time (时间戳):
 - 日志事件发生的时间。
- Record (记录):
 - 一个 JSON 对象, 包含日志的实际内容,
 - 如 `{"message": "user logged in", "level": "info", "user_id": 123}`。

FluentD

- Fluentd is an open source data collector, which lets you unify the data collection and consumption for a better use and understanding of data.



THANK YOU

汇报人：北交大软件学院

严禁复制



SOFAStack 开源家族

